

LuizaLabs - Desafio Técnico - Vertical Logística

Você está na etapa do desafio técnico. Parabéns por ter chegado até aqui!

Neste desafio, queremos conhecer suas habilidades técnicas com um foco prático e aplicado na resolução de um problema com nuances muito próximas do seu dia a dia em nosso time. Aqui conheceremos seu estilo de código, aptidões técnicas, sua capacidade de desenho de arquitetura e, sobretudo, sua habilidade de resolução de problemas e entrega de valor.

O Desafio

Temos uma demanda crítica para integrar dois sistemas logísticos. O sistema legado exporta um arquivo de pedidos totalmente desnormalizado, e nós precisamos transformá-lo em um *payload* JSON estruturado e normalizado. Para isso, precisamos satisfazer alguns requisitos operacionais e de negócio.

Objetivo do Desafio

Faça um sistema que receba um arquivo via API REST e processe-o para ser retornado via API REST.



Cenário de Alta Volumetria (Requisito de Escalabilidade):

O faturamento não pode parar. Sua API deve estar preparada para receber e processar arquivos grandes (ex: até 2GB). Avaliaremos a eficiência da sua solução no consumo de recursos (CPU e Memória) durante a leitura e transformação dos dados.

Entrada de Dados

O arquivo do sistema legado possui uma estrutura em que cada linha representa uma parte de um pedido. Os dados estão padronizados por tamanho de seus valores, respeitando a seguinte tabela:

campo	tamanho	tipo
id usuário	10	numérico
nome	45	texto
id pedido	10	numérico
id produto	10	numérico
valor do produto	12	decimal
data compra	8	numérico (formato: yyyymmdd)

Observações importantes:

- Todos os campos numéricos são completados com '0' à esquerda.
- Os demais campos com espaço à esquerda.
- A formatação das colunas sempre será igual.
- **Bônus de Resiliência:** No mundo real, integrações legadas falham. Preveja e trate como sua aplicação se comportaria caso o arquivo contenha linhas corrompidas (ex: letras no lugar de números no valor do produto).

Dados de exemplo (a primeira linha de cabeçalho não consta no arquivo real):

```
| -userId-- | -----userName----- | -orderId- | -prodId-- | ---value--- | -date-- |
0000000002      Medeiros00000123450000000111      256.2420201201
0000000001      Zarelli00000001230000000111      512.2420211201
0000000001      Zarelli00000001230000000122      512.2420211201
0000000002      Medeiros00000123450000000122      256.2420201201
```

Saída de Dados

A saída de dados deverá ser disponibilizada via API REST, considerando a seguinte estrutura base de *payload* de *response* JSON perfeitamente formatada:

```
[
  {
    "user_id":1,
    "name":"Zarelli",
    "orders":[
      {
        "order_id":123,
        "total":"1024.48",
        "date":"2021-12-01",
        "products":[
          {
            "product_id":111,
            "value":"512.24"
          },
          {
            "product_id":122,
            "value":"512.24"
          }
        ]
      }
    ]
  },
  {
    "user_id":2,
    "name":"Medeiros",
    "orders":[
      {
        "order_id":12345,
        "total":"512.48",
        "date":"2020-12-01",
        "products":[
          {
            "product_id":111,
            "value":"256.24"
          },
          {
            "product_id":122,
            "value":"256.24"
          }
        ]
      }
    ]
  }
]
```

Considere a consulta geral de pedidos e, também, a inclusão de filtros por:

- ID do pedido;
- Intervalo de data de compra (data início e data fim).

Key Words

- Testes Automatizados (Ex: Unitários, Integração, Coverage)
- Lógica & Eficiência
- Simplicidade & Código Limpo
- SOLID
- Linguagem de sua preferência (não estamos focados no framework, mas na engenharia)
- Desenho de API
- Produtividade / AI-Assisted Development
- Processamento em Streams / Alta Volumetria

TL;DR (O que esperamos no seu repositório)

A modelagem e a arquitetura do sistema ficam a seu critério, bem como a seleção e o uso de frameworks e linguagens. É muito importante focar na simplicidade.

- **Documentação:** Deixe claro no README.md as escolhas utilizadas no que tange à tecnologia, gerenciamento de memória e padrões arquiteturais aplicados na resolução.
- **Execução:** Independente da solução, é obrigatório colocar no README o passo a passo claro de como rodar o seu projeto.
- **Persistência:** Sobre o armazenamento para o retorno da API REST, você pode usar a maneira mais benéfica e performática que encontrar (exemplo: arquivo, banco de dados relacional/NoSQL em memória, etc.).
- **Uso de IA:** Caso você tenha utilizado assistentes como GitHub Copilot, Cursor, Claude, entre outros, dedique uma seção do README para nos contar brevemente como a IA apoiou o seu *workflow* (ex: especificação, geração de testes, regex ou modelagem do JSON).

O mais legal desse desafio é ver a sua versatilidade e a lógica implementada na leitura, tratamento e escala dos dados. Boa sorte!